Now we can zoom into our map and pan around, as shown in figure 11.3. If you expect your users to be on browsers that handle SVG very well, like Chrome or Safari, and you don't expect to put more features on a map, then you may not even need to worry about optimization.

But what if you want to build interactive websites that work on all modern browsers? Firefox doesn't have the best SVG performance, and zooming this map in Firefox isn't a pleasant experience. If you change your `d3.range()` setting from 1000 to 5000, then even browsers that handle SVG well start to slow down.

11.1.2 Drawing geodata with canvas

One solution for optimization, which we touched on earlier, is to draw the elements with canvas instead of SVG. That's why we have a canvas element in our sample HTML page for this chapter, and why it's styled in such a way as to be directly underneath our `<svg>` element. Instead of creating SVG elements using D3's enter syntax, we use the built-in functionality in `d3.geo.path` to provide a context for HTML5 canvas. In the following listing, you can see how to use that built-in functionality with your existing dataset.

Listing 11.6 Drawing the map with canvas

```
function createMap(countries) {
  var projection = d3.geo.mercator().scale(50).translate([150,100]);
  var geoPath = d3.geo.path().projection(projection);

  var mapZoom = d3.behavior.zoom().translate(projection.translate())
            .scale(projection.scale()).on("zoom", zoomed);

  d3.select("svg").call(mapZoom);

  zoomed();

  function zoomed() {

    projection.translate(mapZoom.translate()).scale(mapZoom.scale());

    var context = d3.select("canvas").node().getContext("2d");
    context.clearRect(0,0,500,500);
    geoPath.context(context);

    context.strokeStyle = "black";
    context.fillStyle = "gray";
    context.lineWidth = "1px";
    for (var x in countries.features) {
      context.beginPath();
  geoPath(countries.features[x]);
      context.stroke()
      context.fill();
    }

    context.strokeStyle = "black";
    context.fillStyle = "rgba(255,0,0,.2)";
    context.lineWidth = "1px";
    for (var x in sampleData) {
```

Always clear the canvas before redrawing it if you're updating it.

Switches geoPath to a context generator with our canvas context

Styles settings for countries

Draws each country feature to canvas